

Inhaltsverzeichnis

7	Test und Validierung	2
7.1	Validierung des Avisierungsservice-Backends	2
7.1.1	Teststrategie, Anforderungsbezug und Testumgebung	2
7.1.2	Domänen- und Anwendungslogik	4
7.1.3	REST-Verträge, Eingabevalidierung und Fehlerantworten	5
7.1.4	Zugriffsschutz, Sitzungsverwaltung und Mandantenisolation	6
7.1.5	Persistenz- und Abfragemodell	7
7.1.6	Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien	8
7.1.7	Ausgehende Kommunikation und Adapterverträge	9
7.1.8	Ergebnisse und Aussagegrenzen der Backend-Tests	9
7.2	Validierung des Frontends	10

7 Test und Validierung

7.1 Validierung des Avisierungsservice-Backends

Die Validierung des Avisierungsservice-Backends überprüft ausgewählte fachliche und technische Eigenschaften der in Kapitel 5 beschriebenen Lösung. Dabei steht nicht die Prüfung einzelner Klassen im Vordergrund, sondern die Frage, ob zentrale Anforderungen und Entwurfsentscheidungen unter geeigneten Prüfbedingungen bestätigt werden können.

Dazu werden unterschiedliche Testebenen eingesetzt. Fachliche Regeln können weitgehend isoliert geprüft werden, während beispielsweise Mandantentrennung, Datenbankabfragen oder die Ausführung des BPMN-Prozesses das Zusammenspiel mehrerer Komponenten erfordern.

7.1.1 Teststrategie, Anforderungsbezug und Testumgebung

Die Auswahl des Prüfaufbaus richtet sich danach, welche Eigenschaft untersucht werden soll. Dabei werden nur diejenigen technischen Komponenten einbezogen, die für die Aussagekraft des jeweiligen Tests erforderlich sind.

Für isolierte Prüfungen werden nicht relevante Abhängigkeiten durch Test Doubles ersetzt. REST-Verträge werden mit `MockMvc` geprüft. Sicherheitsrelevante Szenarien verwenden einen erweiterten Spring-Boot-Testkontext mit aktivierter Security-Konfiguration und Persistenz.

Datenbankbezogene Prüfungen werden mit PostgreSQL über Testcontainers durchgeführt. Das Schema wird mit den Flyway-Migrationen aufgebaut, sodass JPA-Mappings, Datenbankrestriktionen und QueryDSL-Abfragen gegen dieselbe Datenbanktechnologie wie im Backend geprüft werden.

Für die Workflow-Validierung wird die ausführbare BPMN-Prozessdefinition in den Testkontext eingebunden. Die automatische Camunda-Jobausführung ist deaktiviert, sodass Jobs und zeitabhängige Abläufe gezielt und reproduzierbar ausgeführt werden können. Externe Benachrichtigungs- und DISPO-Komponenten werden dabei durch Test Doubles ersetzt, da der interne Workflow und nicht die Verfügbarkeit externer Systeme geprüft wird.

Die automatisierten Backend-Tests werden im Rahmen des Maven-Builds ausgeführt. Die folgenden Abschnitte betrachten die einzelnen Prüfbereiche anhand ausgewählter Szenarien genauer.

Tabelle 1: Prüfbereiche und zugehöriger Prüfaufbau

Bereich	Repräsentative Prüffälle	Erwartetes Verhalten	Prüfaufbau
Domänen- und Anwendungslogik	abgelaufene Anfrage; identischer Avisierungsauftrag; unzulässiger Zustandswechsel	Fachregeln werden eingehalten und ungültige Zustandsänderungen verhindert	Domain- und Service-Tests mit kontrollierten Abhängigkeiten
REST-Verträge	fehlende Pflichtangaben; ungültige Eingaben; fachlich abgelehnte Requests	definierte HTTP-Statuscodes und Fehlerantworten	MVC-Tests mit <code>MockMvc</code>
Zugriffsschutz und Mandantentrennung	ungültiger API-Key; Zugriff auf fremde Daten; geschützter Dashboard-Zugriff	unzulässige Zugriffe werden verhindert und Mandantengrenzen eingehalten	Spring-Boot-Integrationstests mit Security und Persistenz
Persistenz und Abfragen	aktuelle und historische Anfragen; Filterung nach Tour, Status und Zeitraum	Daten werden korrekt gespeichert, ausgewählt, gruppiert und gefiltert	PostgreSQL-, JPA- und QueryDSL-Integrationstests
Workflow und Nebenläufigkeit	Versand; Fristablauf; Wiederholungen; parallele Aktionen; veraltete Anfrage	Prozesspfade werden korrekt ausgeführt und Zustände bleiben konsistent	Camunda- und Concurrency-Integrationstests
Ausgehende Kommunikation	Benachrichtigungsversand; DISPO-Callback; Fehler externer Abhängigkeiten	ausgehende Aufrufe und Fehlerverarbeitung entsprechen der vorgesehenen Backend-Logik	Adapter- und Integrationstests mit kontrollierten Antworten

7.1.2 Domänen- und Anwendungslogik

Die Domain- und Service-Tests prüfen ausgewählte fachliche Invarianten und Entscheidungsregeln des Avisierungsservice. Im Mittelpunkt stehen zeitliche Grenzbedingungen, Zustandsänderungen sowie die Wiederverwendung, Aktualisierung oder Ersetzung bestehender Avisierungsanfragen.

Zeitabhängige Regeln werden gezielt an ihren Grenzen geprüft. Ein Lieferfenster muss bei der Übernahme noch in der Zukunft liegen, und eine Kundenreaktion ist nur vor Ablauf der Antwortfrist zulässig. Betrachtet werden daher Zeitpunkte unmittelbar vor sowie beim Erreichen der jeweiligen Grenze.

Bei Rückbestätigungsanfragen werden insbesondere unzulässige Zustandsänderungen geprüft. Eine bereits als **SENT** markierte Anfrage darf weder erneut in diesen Zustand überführt noch nachträglich als Zustellfehler gekennzeichnet werden. Änderungen der Auftragsdaten verändern außerdem nicht automatisch den fachlichen Bestätigungsstatus.

Für erneut übermittelte Avisierungsaufträge wird geprüft, ob bestehende Anfragen weiterverwendet, aktualisiert oder ersetzt werden. Eine noch nicht versendete Anfrage kann bei geänderten Daten aktualisiert werden. Bei einer aktiven **SENT**-Anfrage wird dagegen die bestehende Anfrage deaktiviert, ihre Ablösung gemeldet und eine Neuerzeugung mit den aktualisierten Daten veranlasst. Unveränderte laufende oder bereits bestätigte beziehungsweise abgelehnte Aufträge werden wiederverwendet, während nach Versandfehler oder **NO_RESPONSE** auch bei unveränderten Daten eine neue Anfrage entstehen kann.

Bei Kundenreaktionen werden neben Bestätigung und Ablehnung auch ungültige Fälle betrachtet. Antworten auf inaktive, abgelaufene, veraltete oder bereits beantwortete Anfragen werden zurückgewiesen. Für die Duplikatkontrolle wird eine bereits vorhandene Kundenreaktion simuliert. Dabei bleiben Auftragsstatus und Aktivitätszustand unverändert; weder die Speichermethode des Repositorys noch der Workflow-Port werden erneut aufgerufen.

Der Fristablauf darf den Auftrag nur dann auf **NO_RESPONSE** setzen, wenn die Anfrage noch aktiv und die Antwortfrist erreicht ist. Bei einem vorzeitigen Timeout oder einer bereits inaktiven Anfrage erfolgt keine Statusänderung.

Auch die manuelle erneute Avisierung wird anhand ihrer fachlichen Voraussetzungen geprüft. Sie ist nach Versandfehler oder **NO_RESPONSE** zulässig, nicht jedoch bei noch ausstehenden, aktiven oder bereits beantworteten Anfragen. Zusätzlich wird geprüft, dass ein unzulässiger Zustand oder ein bereits begonnenes Lieferfenster vor nachgelagerten Bedingungen erkannt wird. Bei einer Ablehnung bleibt der bestehende Auftragsstatus unverändert und es wird keine neue Anfrage veranlasst.

Der folgende Abschnitt betrachtet die Eingabvalidierung sowie die Abbildung fachlicher Ergebnisse und Fehler auf den REST-Vertrag.

7.1.3 REST-Verträge, Eingabvalidierung und Fehlerantworten

Die Prüfung der REST-Schnittstellen untersucht, ob HTTP-Anfragen korrekt verarbeitet und Ergebnisse der Anwendungsschicht konsistent in HTTP-Antworten überführt werden. Dafür werden sowohl isolierte MVC-Tests mit ersetzten Anwendungsfällen als auch Spring-Boot-Integrationstests eingesetzt. Während MVC-Tests vor allem Parameterbindung, Validierung und Controller-Mapping prüfen, beziehen Integrationstests zusätzlich ausgewählte Teile der Anwendungslogik und Persistenz ein.

Für gültige Dashboard-Abfragen wird geprüft, ob Statusfilter, Tournummern und Datumsgrenzen korrekt in das Abfrageobjekt übernommen werden. Ungültige Parameter wie ein unbekannter Statuswert oder ein nicht ISO-konformes Datum führen zu **400 Bad Request**, ohne dass der Anwendungsfall aufgerufen wird.

Zusätzlich wird die DTO-Validierung betrachtet. Fehlt beispielsweise in einem syntaktisch gültigen JSON-Request das Pflichtfeld `responseType`, wird dieser mit **400 Bad Request** und `VALIDATION_ERROR` zurückgewiesen. Fachlich ungültige Daten werden dagegen durch die Anwendungslogik geprüft. Ein Lieferfenster in der Vergangenheit führt ebenfalls zu **400 Bad Request**; anschließend sind weder Auftrag noch Avisierungsanfrage gespeichert.

Davon zu unterscheiden ist die Abbildung bereits festgestellter fachlicher Fehler. Im MVC-Test für einen ungültigen Dashboard-Zeitraum wird der ersetzte Anwendungsfall so konfiguriert, dass er eine `InvalidFilterException` auslöst. Geprüft wird deren Abbildung auf **400 Bad Request** mit dem Fehlercode `VALIDATION_ERROR`.

Auch erfolgreiche und fachlich abgelehnte Ergebnisse werden auf HTTP-Ebene geprüft. Ein übernommener DISPO-Auftrag mit Status `OPEN` liefert **202 Accepted**. Der Status bestätigt lediglich die Annahme zur weiteren Verarbeitung, nicht den Abschluss des Avisierungsprozesses. Eine gültige Kundenreaktion führt zu **204 No Content**, eine abgelaufene Anfrage zu **410 Gone** und eine inaktive beziehungsweise bereits verarbeitete Anfrage zu **409 Conflict**.

Für die Dashboard-Detailansicht wird außerdem geprüft, ob Auftragsdaten, aktuelle Avisierungsanfrage, Historie und Kundenreaktion in der vorgesehenen JSON-Struktur ausgegeben werden. Fehlerantworten enthalten einen maschinenlesbaren Fehlercode, eine Beschreibung, HTTP-Status, Pfad und Zeitstempel. Für einen nicht vorhandenen Auftrag werden alle fünf Felder geprüft; weitere Szenarien kontrollieren insbesondere die Zuordnung von Status und Fehlercode. Die gemeinsame Struktur wird durch den `GlobalExceptionHandler` erzeugt.

Auch die E-Mail-Einstellungen prüfen weitere Vertragseigenschaften. Fehlt eine SMTP-Konfiguration, wird **200 OK** mit `configured=false` zurückgegeben. Passwort und verschlüsselte Repräsentation dürfen nicht Bestandteil der Antwort sein. Werden die ersetzten Anwendungsfälle für Verbindungstest oder Testnachricht mit den vorgesehenen Ausnahmen

konfiguriert, wird deren Abbildung auf 502 Bad Gateway geprüft.

Die Aussagekraft ist auf die jeweils ausgeführten Komponenten begrenzt. In den MVC-Slice-Tests sind Anwendungsfälle ersetzt und Security-Filter deaktiviert. Integrationstests beziehen ausgewählte Anwendungs- und Persistenzkomponenten ein, ersetzen jedoch ebenfalls bestimmte Abhängigkeiten durch Test Doubles. Abschnitt 7.1.4 betrachtet daher ergänzend Zugriffsschutz, Sitzungsverwaltung und Mandantenisolation.

7.1.4 Zugriffsschutz, Sitzungsverwaltung und Mandantenisolation

Während Abschnitt 7.1.3 die Verarbeitung von HTTP-Anfragen und deren Abbildung auf den REST-Vertrag untersucht, betrachtet dieser Abschnitt die Durchsetzung von Zugriffsregeln und Mandantengrenzen. Hierfür werden Integrationstests mit aktiver Sicherheitskonfiguration und PostgreSQL eingesetzt.

Für die DISPO-Schnittstelle wird geprüft, dass fehlende oder ungültige Zugangsdaten zurückgewiesen und gültige Zugangsdaten dem richtigen Unternehmen zugeordnet werden. Gleichzeitig wird sichergestellt, dass die für die System-zu-System-Kommunikation verwendeten Zugangsdaten keinen direkten Zugriff auf geschützte Dashboard-Funktionen ermöglichen.

Der Dashboard-Zugriff wird über einen zeitlich begrenzten Zugangscode in eine authentifizierte Sitzung überführt. Die Tests prüfen, dass ein gültiger Code erfolgreich verwendet werden kann, während eine erneute Verwendung oder ein bereits abgelaufener Code zurückgewiesen wird.

Für geschützte Dashboard-Operationen wird zusätzlich der CSRF-Schutz überprüft. Ändernde Anfragen ohne gültigen CSRF-Nachweis werden abgewiesen, bevor die zugehörige Anwendungslogik ausgeführt wird. Bei gültiger Authentifizierung und gültigem CSRF-Nachweis wird die vorgesehene Operation zugelassen.

Der Kundenzugriff erfolgt unabhängig von DISPO- und Dashboard-Authentifizierung über einen individuellen Zugriffstoken. Die Tests prüfen, dass gültige Tokens die zugehörigen Bestätigungsdaten bereitstellen, unbekannte oder durch eine neuere Anfrage abgelöste Tokens dagegen keinen Zugriff mehr ermöglichen.

Ein weiterer Schwerpunkt liegt auf der Mandantenisolation. Hierzu werden Daten mehrerer Unternehmen parallel angelegt und anschließend geprüft, dass Lese- und Änderungsoperationen jeweils auf die Daten des authentifizierten Unternehmens beschränkt bleiben. Fremde Aufträge können nicht verändert werden. Dieselbe Trennung wird zusätzlich für unternehmensspezifische E-Mail-Konfigurationen überprüft.

Die Tests bestätigen damit die Trennung der verschiedenen Zugriffsmodelle sowie die Mandantenisolation für ausgewählte Lese- und Schreiboperationen. Reales Browserverhalten, Cookie-Eigenschaften, der konfigurierte Inaktivitätstimeout sowie einzelne technische Details der Sitzungsverwaltung werden durch diese Szenarien nicht überprüft. Der folgende Abschnitt betrachtet die Persistenz und die leseseitigen Abfragen.

7.1.5 Persistenz- und Abfragemodell

Die Persistenz- und Abfragetests prüfen, ob Avisierungsdaten entsprechend dem Datenmodell gespeichert und für Tourübersicht und Detailansicht korrekt rekonstruiert werden. Als Testdatenbank wird PostgreSQL über Testcontainers verwendet. Das Schema wird durch Flyway aufgebaut und anschließend von Hibernate gegen die JPA-Abbildungen validiert. Vor den eigentlichen Abfragen werden ausstehende Änderungen mit `flush` in die Datenbank geschrieben und der Persistence Context geleert. Dadurch basieren die Ergebnisse auf tatsächlich persistierten Daten.

Eine zentrale Regel des Abfragemodells besteht darin, für einen Auftrag stets die Avisierungsanfrage mit der höchsten internen ID als aktuelle Anfrage zu verwenden. Die Tests prüfen, dass ein älterer Token nach einer neueren Anfrage nicht mehr aufgelöst werden kann, während der Token der aktuellen Anfrage weiterhin gültig ist. Diese Auswahl erfolgt jeweils innerhalb des zugehörigen Auftrags. Auch Tourübersicht und Detailansicht verwenden diese Regel, sodass Lieferdatum, Lieferfenster und Kommunikationskanal aus der jeweils aktuellen Anfrage stammen.

Für die Tourübersicht werden Filterung und Suche separat betrachtet. Datums Grenzen werden einschließlich ihrer Randwerte ausgewertet. Unterschiedliche Filterkriterien werden miteinander verknüpft, während mehrere Werte innerhalb eines Kriteriums alternativ gelten. Zusätzlich wird zwischen Suchtreffern auf Tour- und Auftragsebene unterschieden. Ein Treffer in Tournummer oder Kennzeichen lässt die Tour grundsätzlich im Ergebnis, während weitere Filter weiterhin auf die enthaltenen Aufträge wirken. Ein Treffer ausschließlich in auftragsbezogenen Feldern führt dagegen dazu, dass nur die passenden Aufträge innerhalb der Tour angezeigt werden. Die Suche ist unabhängig von Groß- und Kleinschreibung.

Für Gruppierung und Paginierung gilt die fachliche Vorgabe, dass eine Tournummer nicht für unterschiedliche Liefertage oder Fahrzeuge wiederverwendet wird. Aufträge derselben Tour werden daher gemeinsam dargestellt. Die Touren werden nach Lieferdatum und Tournummer sortiert, die enthaltenen Aufträge nach Beginn und Ende des Lieferfensters sowie nach externer Auftrags-ID. Die Paginierung erfolgt auf Tour-Ebene und trennt damit keine Aufträge derselben Tour. In einem Testszenario werden drei Touren bei einer Seitengröße von zwei auf zwei Seiten verteilt; sämtliche Aufträge der ersten Tour bleiben dabei gemeinsam erhalten.

Für die Detailansicht wird geprüft, ob die aktuelle Anfrage von ihrer Historie korrekt getrennt wird. Die neueste Anfrage erscheint als `currentRequest`, ältere Anfragen werden in absteigender Reihenfolge ihrer ID als `previousRequests` ausgegeben. Diese Zuordnung hängt nicht davon ab, ob die aktuelle Anfrage bereits versendet wurde oder noch den technischen Status `PENDING` beziehungsweise `FAILED` besitzt. Vorhandene Kundenreaktionen werden mit Typ, Kommentar und Eingangszeitpunkt übernommen. Für Aufträge ohne Avisierungsanfragen wird ein Ergebnis ohne `currentRequest` und mit leerer Historie

geliefert; bei einem unbekannten Auftrag wird kein Detailergebnis zurückgegeben. Die Mandantenisolation wurde bereits in Abschnitt 7.1.4 betrachtet.

Die Tests bestätigen damit die wesentlichen Persistenz- und Abfragerregeln unter Verwendung derselben Datenbanktechnologie wie im Backend.

7.1.6 Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien

Die Workflow-Tests prüfen ausgewählte Pfade der ausführbaren BPMN-Prozessdefinition sowie das Verhalten bei Wiederholungen und konkurrierenden Operationen. Die automatische Job-Ausführung ist dabei deaktiviert, sodass einzelne Jobs und Timer gezielt ausgeführt und die daraus entstehenden Prozess- und Datenbankzustände überprüft werden können. Für zeitabhängige Szenarien werden Anwendungs- und Prozesszeit kontrolliert gesetzt.

Für den regulären Prozessablauf wird zunächst geprüft, dass nach dem Prozessstart ein asynchroner Versand-Job vorliegt und noch keine Versandaktion erfolgt ist. Nach dessen erfolgreicher Ausführung müssen Anfrage und Auftrag den erwarteten Versandstatus besitzen. Zusätzlich werden die vorgesehenen Wartezustände des Prozesses sowie der Fälligkeitszeitpunkt des Antworttimers überprüft.

Weitere Tests betrachten die alternativen Prozesspfade. Eine Kundenreaktion muss einen Callback-Job mit den erwarteten Prozessdaten erzeugen und der Prozess nach erfolgreicher Verarbeitung enden. Beim Erreichen der Antwortfrist werden Anfrage und Auftrag auf den vorgesehenen Zustand `NO_RESPONSE` überführt und ebenfalls ein Callback vorbereitet. Wird eine Anfrage durch eine neuere ersetzt, muss die bisherige Prozessinstanz dagegen ohne Callback beendet werden.

Die Wiederholungslogik des Versands wird mit kontrolliert erzeugten Fehlern geprüft. Bei einem wiederholbaren Fehler werden insgesamt bis zu drei Versandversuche durchgeführt. Die Tests überprüfen dabei sowohl die Anzahl der Versuche als auch die konfigurierten Wartezeiten von einer beziehungsweise fünf Minuten. Bleiben alle Versuche erfolglos, wird die Anfrage auf `FAILED` gesetzt und der Prozess ohne DISPO-Callback beendet. Für einen als dauerhaft behandelten Fehler wird geprüft, dass keine weiteren fachlich modellierten Wiederholungsversuche stattfinden.

Davon getrennt wird das Verhalten bei einem unerwarteten technischen Fehler geprüft. In diesem Fall darf kein fachlicher Versandversuch fortgeschrieben werden. Stattdessen wird der technische Retry-Zähler des Camunda-Jobs reduziert, während die Anfrage im vorherigen Zustand verbleibt und kein fachlicher Wiederholungstimer entsteht. Dadurch wird geprüft, dass fachliche und technische Wiederholungsmechanismen voneinander getrennt bleiben.

Auch Fehler beim DISPO-Callback werden untersucht. Nach Ausschöpfung der technischen Job-Retries muss ein `failedJob`-Incident vorliegen, während die Prozessinstanz bestehen bleibt. Ein zusätzlicher Test prüft den Fall, dass die betreffende Avisierungsan-

frage zwischenzeitlich durch eine neuere ersetzt wurde. Der veraltete Callback darf dann nicht mehr ausgeführt werden; die alte Prozessinstanz und der zugehörige Incident werden beendet.

Die Nebenläufigkeitstests prüfen mehrere konkurrierende Zugriffe auf denselben fachlichen Zustand. Bei zwei parallelen Versandaufrufen wird kontrolliert, dass die externe Versandaktion nur einmal erfolgt und anschließend ein konsistenter Versandstatus gespeichert ist. Der Test bezieht sich dabei auf parallele Aufrufe der Anwendungslogik und nicht auf mehrere gleichzeitig arbeitende Camunda Job Executor.

Für gleichzeitig eintreffende Kundenreaktionen **CONFIRM** und **REJECT** wird geprüft, dass genau eine Reaktion gespeichert und nur eine der beiden Operationen erfolgreich abgeschlossen wird. Der resultierende Auftragsstatus muss mit der tatsächlich gespeicherten Reaktion übereinstimmen; welche der konkurrierenden Reaktionen gewinnt, ist nicht festgelegt.

Ein weiterer Nebenläufigkeitstest betrachtet die gleichzeitige Kundenbestätigung und Ablösung einer bestehenden Anfrage. Dabei wird geprüft, dass die beiden Operationen nicht zu einem widersprüchlichen Zwischenzustand führen. Nach Abschluss beider Transaktionen ist genau eine Kundenbestätigung gespeichert, die bisherige Anfrage inaktiv und der Auftrag für die neu erzeugte Avisierung wieder **OPEN**.

7.1.7 Ausgehende Kommunikation und Adapterverträge

Die Tests erfassen damit ausgewählte Eigenschaften der backendseitigen Nachrichtenaufbereitung, Aufrufbildung und Fehlerabbildung.

7.1.8 Ergebnisse und Aussagegrenzen der Backend-Tests

Die Ergebnisse stützen die korrekte Umsetzung zentraler fachlicher Regeln und technischer Schutzmechanismen für die untersuchten Szenarien. Unzulässige Zustandsänderungen werden zurückgewiesen, zeitliche Zulässigkeitsgrenzen berücksichtigt und gültige Kundenreaktionen, Fristabläufe sowie erneute Avisierungen in die vorgesehenen Zustände überführt. REST-Schnittstellen, Zugriffsregeln und Mandantengrenzen verhalten sich in den geprüften Fällen entsprechend den definierten Vorgaben.

Auf Integrationsebene liefern die PostgreSQL-Abfragen die erwarteten Filter-, Gruppierung-, Historien- und Paginierungsergebnisse. Die geprüften BPMN-Pfade verarbeiten Kundenreaktionen, Fristabläufe, Versandwiederholungen und abgelöste Anfragen wie vorgesehen. In den untersuchten Konkurrenzsituationen bleiben die geprüften Konsistenzbedingungen erhalten. Die ausgehenden Adapter erzeugen die erwarteten Nachrichten und HTTP-Aufrufe und bilden ausgewählte technische Fehler auf die vorgesehene Backend-Logik ab.

Die Aussagekraft ist auf die beschriebenen Prüfbedingungen begrenzt. Externe Systeme wie DISPO, SMTP-Provider oder Twilio werden teilweise durch Test Doubles oder

simulierte Antworten ersetzt. Geprüft werden damit die backendseitigen Adapterverträge, nicht die tatsächliche Verfügbarkeit oder das Verhalten produktiver Fremdsysteme. Reales Browserverhalten, Cookie-Eigenschaften und der konfigurierte Inaktivitätstimeout werden ebenfalls nicht abgedeckt.

Camunda-Jobs und Timer werden bei deaktivierter automatischer Jobausführung gezielt ausgeführt. Daher werden kontrollierte Prozessabläufe geprüft, nicht das vollständige Betriebsverhalten der automatischen Jobausführung. Ebenfalls nicht untersucht werden Last- und Performanzverhalten, die Migration bestehender Produktivdatenbanken, mehrere parallel betriebene Backend-Instanzen oder beliebige Ausfallszenarien. Die Nebenläufigkeitstests decken nur ausgewählte Konkurrenzsituationen ab.

Eine weitere Grenze betrifft externe Seiteneffekte. Da Datenbankänderungen und externe Kommunikationsaufrufe nicht atomar gekoppelt sind, kann eine erneute Ausführung zu wiederholten externen Effekten führen. Die implementierten Zustandsprüfungen reduzieren dieses Risiko, gewährleisten jedoch keine globale Exactly-once-Ausführung.

Insgesamt bestätigen die Tests, dass die zentralen fachlichen Regeln, Sicherheitsmechanismen sowie Persistenz-, Workflow- und Integrationsfunktionen des Backends unter den definierten Prüfbedingungen wie vorgesehen zusammenwirken. Damit stützen die Testergebnisse die technische Tragfähigkeit der in Kapitel 5 beschriebenen Backend-Lösung.

7.2 Validierung des Frontends